

Diagram

- Design Pattern
- Design Table ที่ยืดหยุ่น
- Calculation Flow ที่คำนวณ promotion ซ้อนกันได้ถูกต้อง

1. Overall Promotion Engine Design

ภาพนี้ใช้อธิบายภาพรวมระบบว่า Order เข้ามาแล้ว Promotion Engine ทำงานผ่าน Rule, Strategy และ Database อย่างไร

flowchart LR

A["Client / Customer"] --> B["Order API / Pricing API"]

B --> C["Pricing Service"]

C --> D["Load Product Price
จาก products table"]

C --> E["Load Active Promotions
จาก promotions tables"]

D --> F["Build Calculation Context"]

E --> F

F --> G["Promotion Engine"]

G --> H["Condition Evaluator
(Specification Pattern)"]

G --> I["Promotion Sorter
(Scope + Priority)"]

G --> J["Stacking Policy
(stackable / exclusive)"]

G --> K["Strategy Registry
(Factory / Registry Pattern)"]

K --> L["Percentage Discount Strategy"]

```

K --> M["Fixed Amount Discount Strategy"]
K --> N["Future Promotion Strategy"]

H --> O["Eligible Promotions"]
I --> O
J --> O

O --> P["Apply Promotions"]
P --> Q["Cap Discount<br/>ห้ามลดเกินราคาที่เหลือ"]
Q --> R["Final Price + Breakdown"]

R --> S["Save Calculation Log"]
S --> T["promotion_calculation_logs"]

R --> U["Return Result"]

```



แนวคิดของภาพนี้

ระบบไม่ควรให้ **Pricing Service** คำนวณ promotion เองทั้งหมด แต่ควรแยกเป็น **Promotion Engine** โดยเฉพาะ เพื่อให้:

- เพิ่ม promotion ใหม่ได้ง่าย
- คุมลำดับ promotion ซ้อนกันได้
- ตรวจสอบย้อนหลังได้
- ไม่ทำให้ logic คำนวณราคากระจายไปหลายที่

2. Promotion Calculation Flow

ภาพนี้ตอบคำถามว่า ถ้ารองรับ promotion ซ้อนกัน ต้องจัดลำดับคำนวณยังไง

flowchart TD

```

A["Start Calculate Price"] --> B["Validate Order Request"]

```

```

t"]
  B --> C["Load Product Price From DB"]
  C --> D["Build Calculation Context"]

  D --> E["Load Active Promotions"]
  E --> F["Filter By Date / Status / Usage Limit"]

  F --> G["Check Target<br/>Product / Category / Cart"]
  G --> H["Check Conditions<br/>Min Amount / Coupon / Segme
nt"]

  H --> I["Sort Promotions"]

  I --> I1["1. ITEM Level"]
  I1 --> I2["2. CART Level"]
  I2 --> I3["3. COUPON Level"]
  I3 --> I4["4. SHIPPING Level"]
  I4 --> I5["5. ROUNDING"]

  I5 --> J["Apply Promotion By Priority ASC"]

  J --> K{"Stackable?"}
  K -- "Yes" --> L["Apply Promotion"]
  K -- "No" --> M{"Conflict With Applied Promo?"}

  M -- "No" --> L
  M -- "Yes" --> N["Skip Promotion<br/>บันทึกเหตุผล"]

  L --> O["Calculate Discount"]
  O --> P["actual_discount = min(calculated_discount, remai
ning_amount)"]
  P --> Q["Update Running Total"]

  Q --> R{"stop_processing = true?"}
  R -- "Yes" --> S["Stop Current Promotion Group"]
  R -- "No" --> T{"More Promotions?"}

```

N --> T

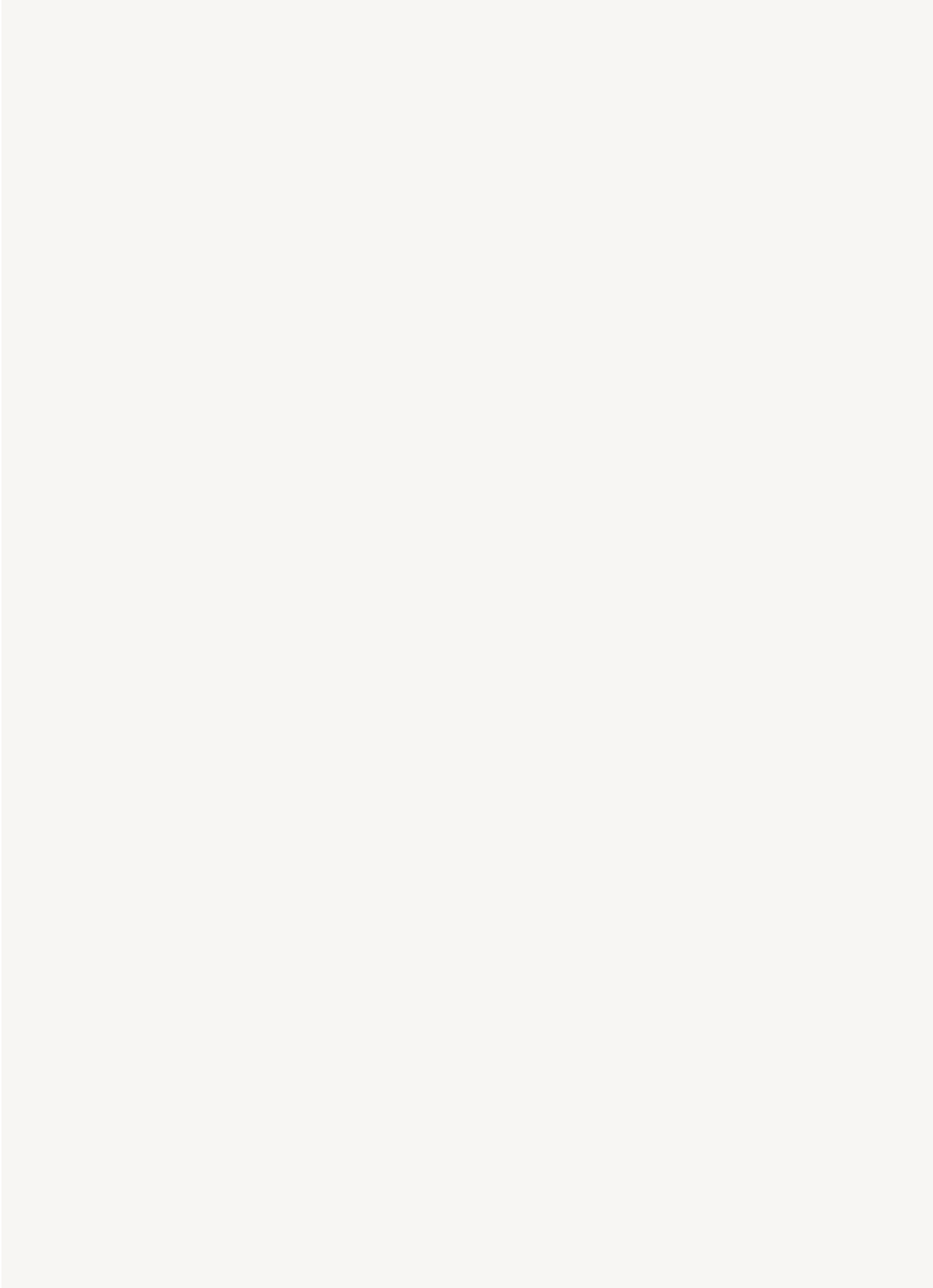
S --> T

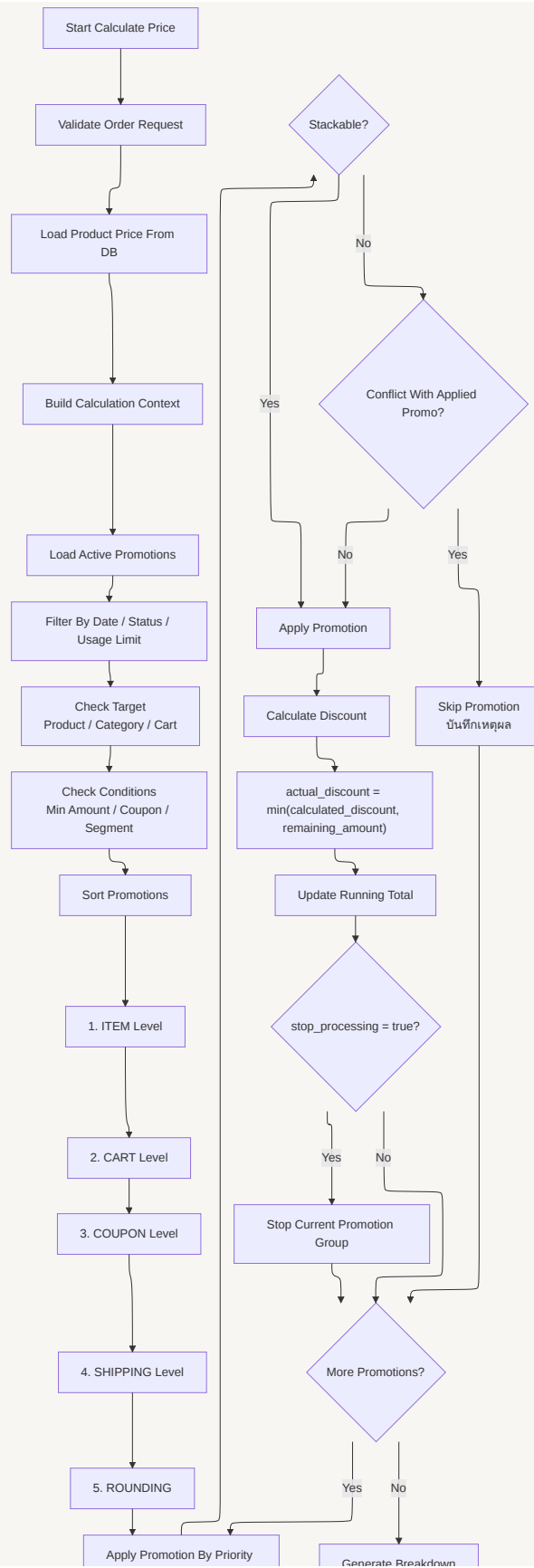
T -- "Yes" --> J

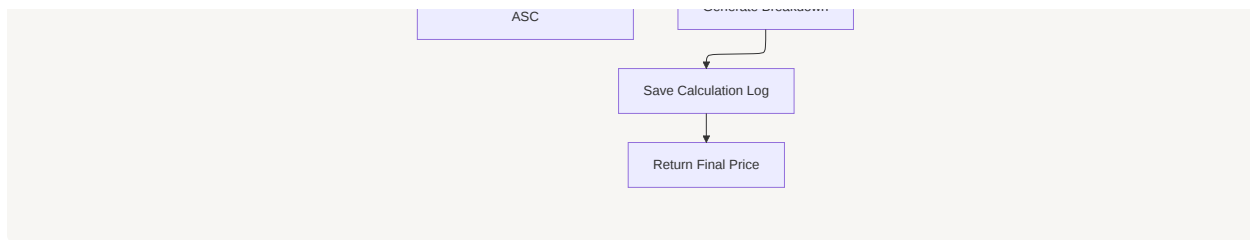
T -- "No" --> U["Generate Breakdown"]

U --> V["Save Calculation Log"]

V --> W["Return Final Price"]







ลำดับที่แนะนำ

ITEM → CART → COUPON → SHIPPING → ROUNDING

เหตุผล

- **ITEM** ต้องมาก่อน เพราะเป็นส่วนลดระดับสินค้า เช่น Product 1 ลด 10%
- **CART** ควรคำนวณจากยอดหลังหัก item discount
- **COUPON** มักเป็นส่วนลดพิเศษที่ใช้หลัง subtotal
- **SHIPPING** แยกจากราคาสินค้า
- **ROUNDING** ทำท้ายสุดเพื่อให้ยอดเงินจริงถูกต้อง

3. Promotion Stacking Policy Diagram

ภาพนี้ใช้อธิบายว่า promotion ซ้อนกันจะถูกเลือกหรือถูก skip อย่างไร

flowchart TD

```

A["Promotion Candidate"] --> B{"Promotion Active?"}
B -- "No" --> X["Skip: Not Active"]
B -- "Yes" --> C{"Within Date Range?"}
C -- "No" --> Y["Skip: Expired / Not Started"]
C -- "Yes" --> D{"Target Matched?"}
D -- "No" --> Z["Skip: Target Not Matched"]
D -- "Yes" --> E{"Condition Passed?"}
E -- "No" --> AA["Skip: Condition Failed"]
  
```

```
E -- "Yes" --> F{"Usage Limit Available?"}

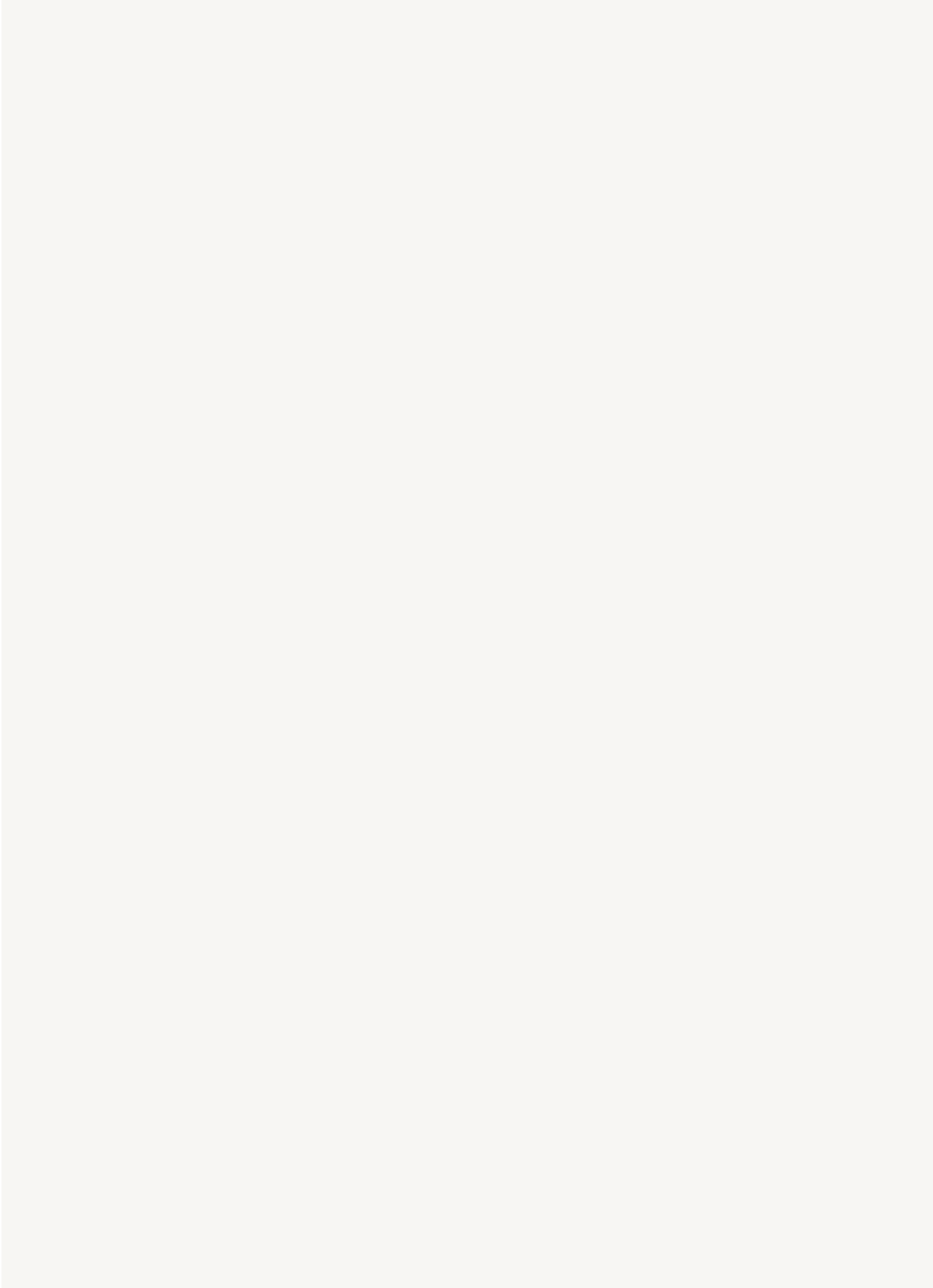
F -- "No" --> AB["Skip: Usage Limit Reached"]
F -- "Yes" --> G{"Conflict With Applied Promotion?"}

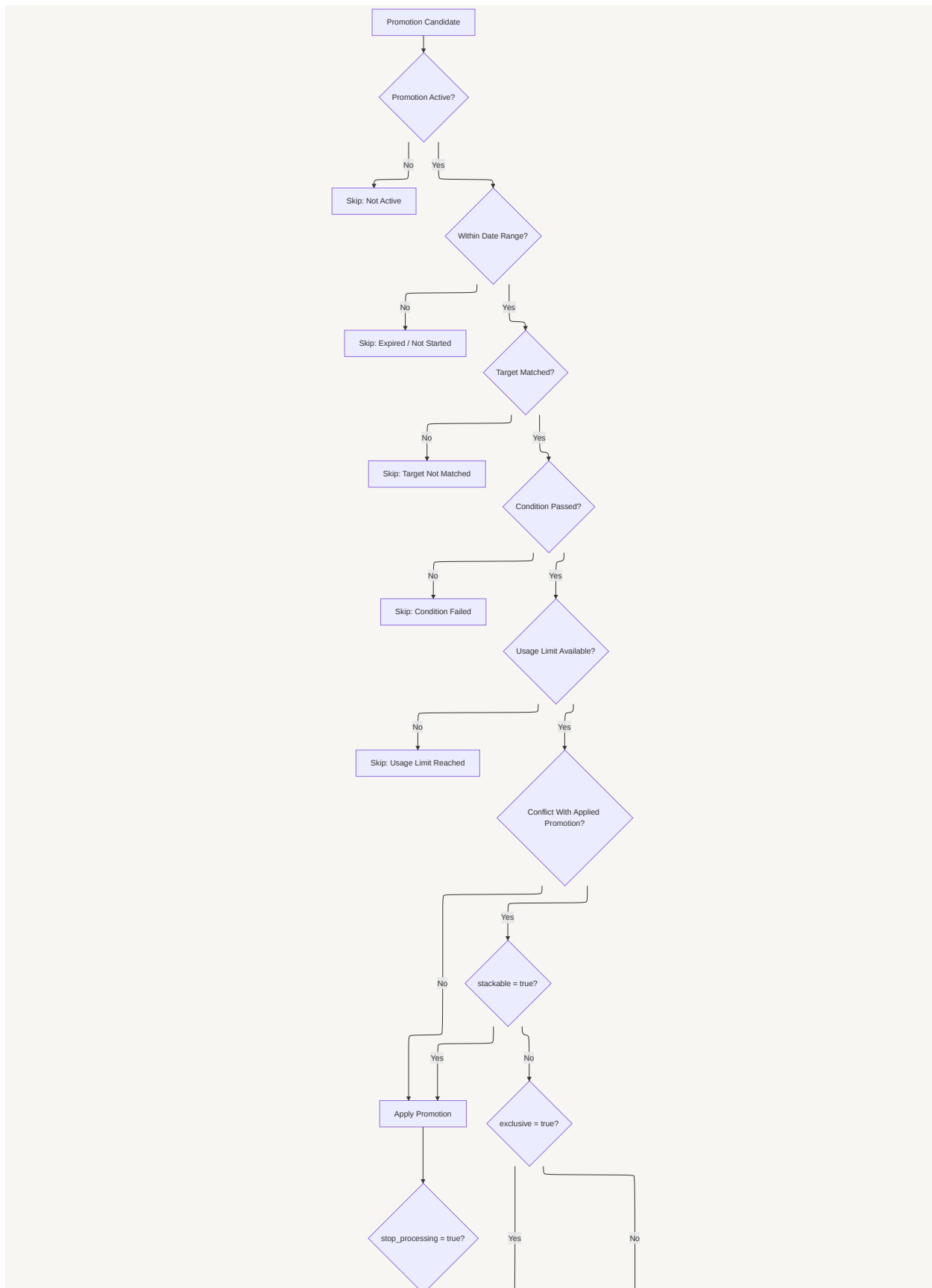
G -- "No" --> H["Apply Promotion"]
G -- "Yes" --> I{"stackable = true?"}

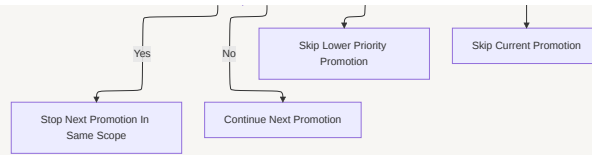
I -- "Yes" --> H
I -- "No" --> J{"exclusive = true?"}

J -- "Yes" --> K["Skip Lower Priority Promotion"]
J -- "No" --> L["Skip Current Promotion"]

H --> M{"stop_processing = true?"}
M -- "Yes" --> N["Stop Next Promotion In Same Scope"]
M -- "No" --> O["Continue Next Promotion"]
```





Field สำคัญที่ใช้ควบคุม

Field	ใช้ทำอะไร
<code>priority</code>	เลขน้อยค่านวนก่อน
<code>stackable</code>	ใช้ร่วมกับ promotion อื่นได้ไหม
<code>exclusive</code>	ถ้าใช้ตัวนี้แล้ว ห้ามตัวอื่นที่ชนกัน
<code>stop_processing</code>	ใช้แล้วหยุด promotion ถัดไปใน scope เดียวกัน
<code>scope</code>	แยก ITEM, CART, COUPON, SHIPPING
<code>max_usage</code>	จำกัดจำนวนการใช้ทั้งหมด
<code>max_usage_per_user</code>	จำกัดจำนวนการใช้ต่อ user

4. Design Pattern Diagram

ภาพนี้ตอบว่า ควรใช้ Design Pattern อะไร และแต่ละตัวทำหน้าที่อะไร

flowchart LR

A["Promotion Engine"] --> B["Specification Pattern"]

A --> C["Strategy Pattern"]

A --> D["Factory / Registry Pattern"]

A --> E["Policy Pattern"]

A --> F["Pipeline Pattern"]

A --> G["Repository Pattern"]

B --> B1["ตรวจว่า Promotion ใช้ได้หรือไม่
Condition / Target / Date / Usage"]

C --> C1["แยกวิธีคำนวณ Promotion แต่ละแบบ"]

C1 --> C2["PercentageDiscountStrategy"]

C1 --> C3["FixedAmountDiscountStrategy"]

C1 --> C4["FreeShippingStrategy"]

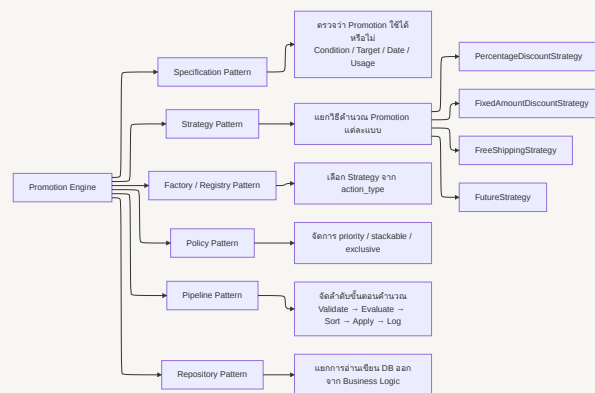
C1 --> C5["FutureStrategy"]

D --> D1["เลือก Strategy จาก action_type"]

E --> E1["จัดการ priority / stackable / exclusive"]

F --> F1["จัดลำดับขั้นตอนคำนวณ
Validate → Evaluate → Sort → Apply → Log"]

G --> G1["แยกการอ่านเขียน DB ออกจาก Business Logic"]



Pattern ที่เหมาะที่สุดสำหรับโจทย์นี้

Rule-based Engine
+ Strategy Pattern
+ Specification Pattern
+ Factory / Registry Pattern
+ Policy Pattern

เหตุผล

- **Strategy Pattern** ทำให้เพิ่มวิธีลดราคาใหม่ได้
- **Specification Pattern** ทำให้เพิ่มเงื่อนไขใหม่ได้
- **Factory / Registry** ทำให้ engine เลือก strategy ได้โดยไม่ต้องเขียน if/else ใหญ่
- **Policy Pattern** ทำให้คุม promotion ซ้อนกันได้

- **Repository Pattern** ทำให้ logic ไม่ผูกกับ MySQL/GORM โดยตรง

5. Flexible Database Design Diagram

ภาพนี้อธิบายว่า DB ถูกออกแบบให้ promotion หนึ่งตัวประกอบจาก **Target + Condition + Action + Policy**

erDiagram

```
product_categories ||--o{ products : "has"
products ||--o{ order_items : "used in"
orders ||--o{ order_items : "contains"

promotions ||--o{ promotion_targets : "has targets"
promotions ||--o{ promotion_conditions : "has conditions"
promotions ||--o{ promotion_actions : "has actions"
promotions ||--o{ promotion_usages : "has usages"

orders ||--o{ promotion_usages : "uses promotion"
orders ||--o{ promotion_calculation_logs : "has calculation logs"

product_categories {
    bigint id PK
    varchar name
    bigint parent_id FK
    enum status
    datetime created_at
    datetime updated_at
    datetime deleted_at
}

products {
    bigint id PK
    varchar sku
    varchar name
    bigint category_id FK
```

```

        bigint price_amount
        varchar currency
        enum status
    }

    orders {
        bigint id PK
        varchar order_no
        bigint user_id
        bigint original_total
        bigint discount_total
        bigint final_total
        varchar currency
        enum status
    }

    order_items {
        bigint id PK
        bigint order_id FK
        bigint product_id FK
        int quantity
        bigint unit_price
        bigint original_amount
        bigint discount_amount
        bigint final_amount
    }

    promotions {
        bigint id PK
        varchar code
        varchar name
        enum scope
        int priority
        boolean stackable
        boolean exclusive
        boolean stop_processing
    }

```

```

        enum status
        datetime starts_at
        datetime ends_at
        int max_usage
        int max_usage_per_user
    }

    promotion_targets {
        bigint id PK
        bigint promotion_id FK
        enum target_type
        bigint target_id
        varchar target_value
    }

    promotion_conditions {
        bigint id PK
        bigint promotion_id FK
        enum condition_type
        enum operator
        json value_json
        varchar group_key
        enum logical_operator
    }

    promotion_actions {
        bigint id PK
        bigint promotion_id FK
        enum action_type
        bigint value_amount
        decimal value_percent
        json value_json
        bigint max_discount_amount
        enum applies_to
    }

```

```

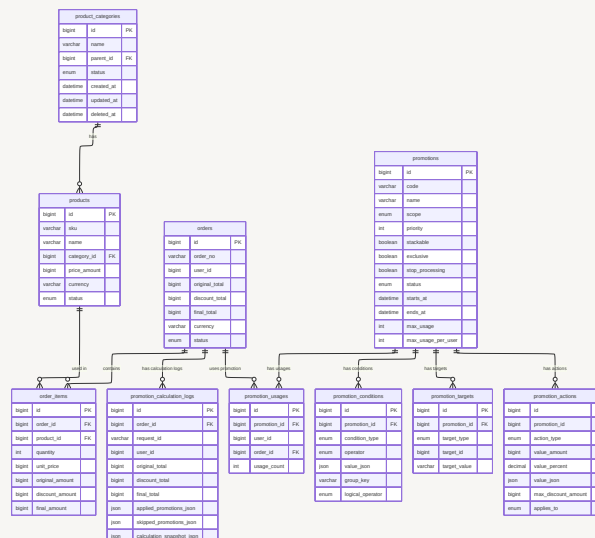
promotion_usages {
    bigint id PK
    bigint promotion_id FK
    bigint user_id
    bigint order_id FK
    int usage_count
}

```

```

promotion_calculation_logs {
    bigint id PK
    bigint order_id FK
    varchar request_id
    bigint user_id
    bigint original_total
    bigint discount_total
    bigint final_total
    json applied_promotions_json
    json skipped_promotions_json
    json calculation_snapshot_json
}

```



ทำไม DB นี้ยืดหยุ่น

เพราะไม่ได้ผูก promotion ไว้เป็น column ตายตัว เช่น `discount_percent`, `discount_amount`, `product_id` ในตารางเดียว แต่แยกเป็น:

```
Promotion Header
→ Target: ใช้กับอะไร
→ Condition: ต้องเข้าเงื่อนไขอะไร
→ Action: ลดอย่างไร
→ Policy: ช้อนกันได้ไหม
```

6. Add Existing Promotion Flow

ภาพนี้ตอบว่า ถ้าต้องการเพิ่ม promotion ที่ระบบรองรับอยู่แล้ว จะทำยังไงไม่ให้เกิด logic เดิม

flowchart TD

```
A["Admin ต้องการเพิ่ม Promotion ใหม่"] --> B["Promotion Type รองรับแล้วหรือยัง?"]
```

```
B -- "รองรับแล้ว<br/>เช่น Percentage / Fixed Amount" --> C["Insert promotions"]
```

```
C --> D["Insert promotion_targets"]
```

```
D --> E["Insert promotion_conditions<br/>ถ้ามี"]
```

```
E --> F["Insert promotion_actions"]
```

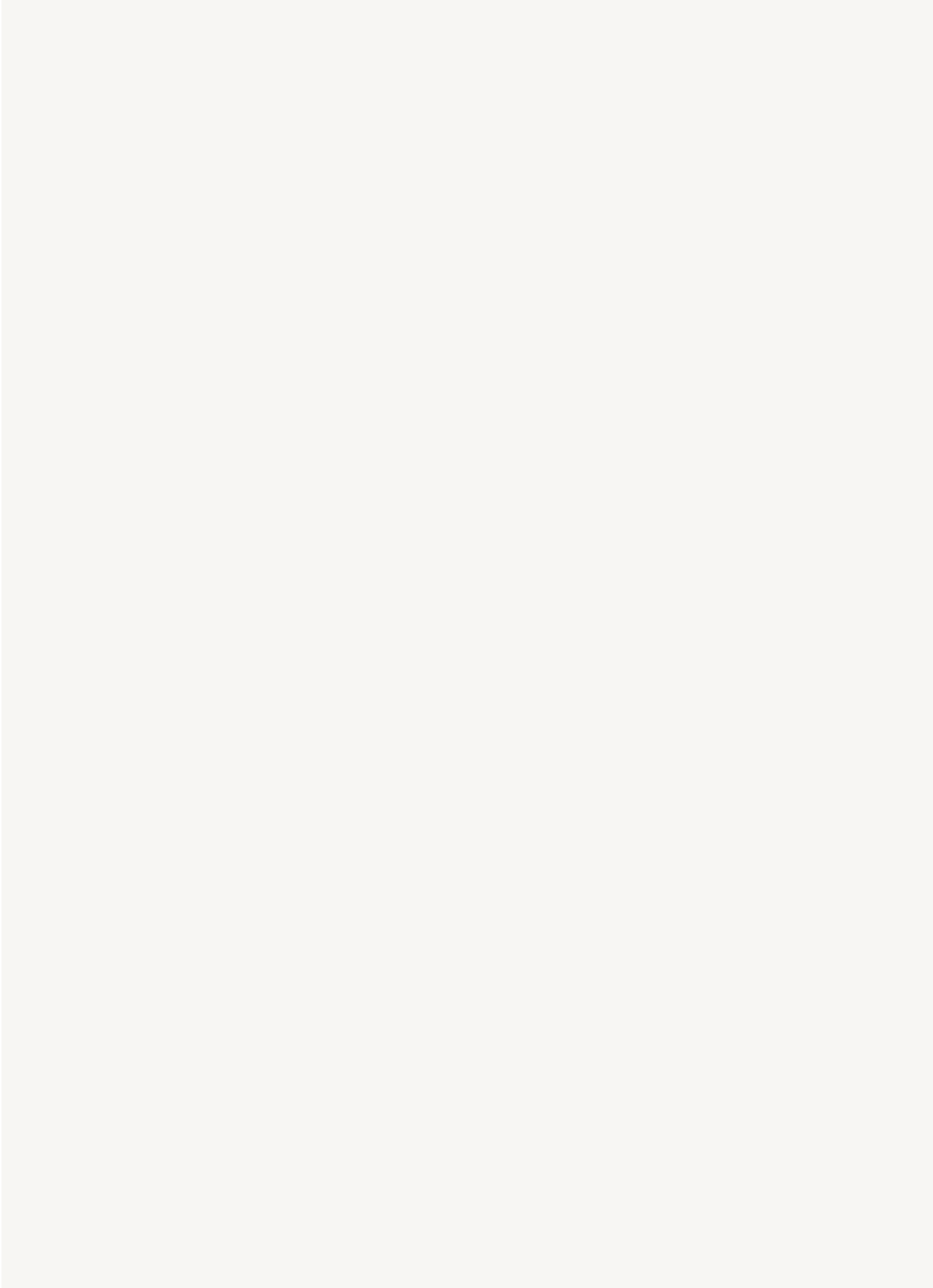
```
F --> G["Validate Promotion Config"]
```

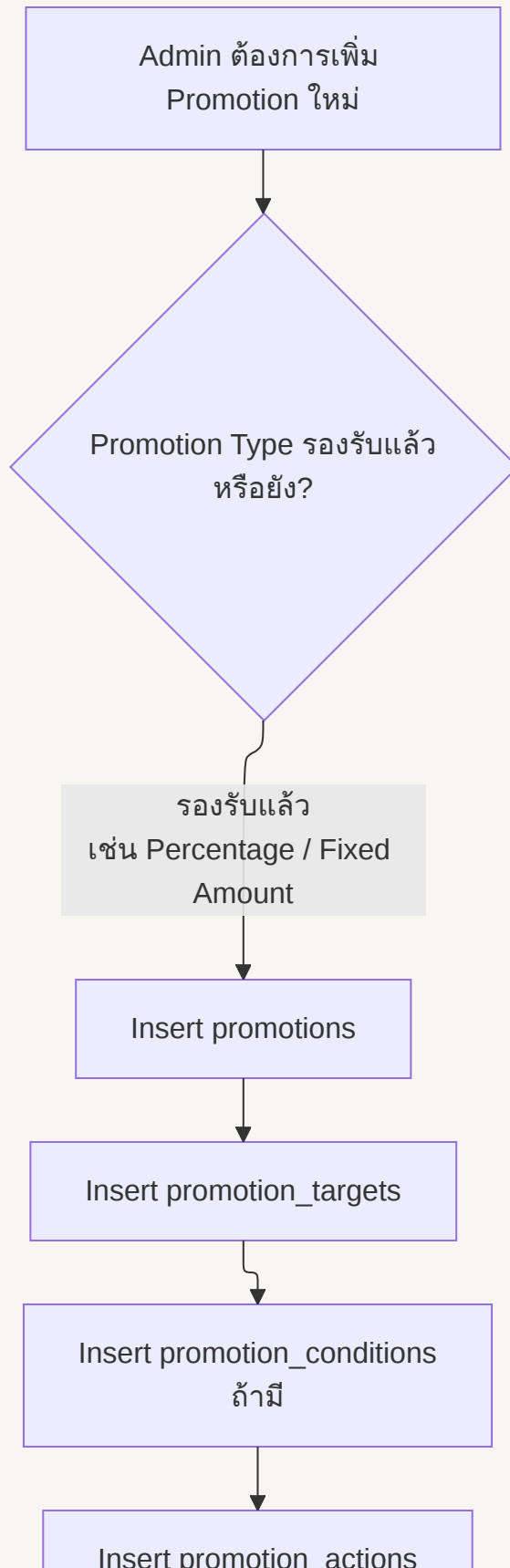
```
G --> H["Activate Promotion"]
```

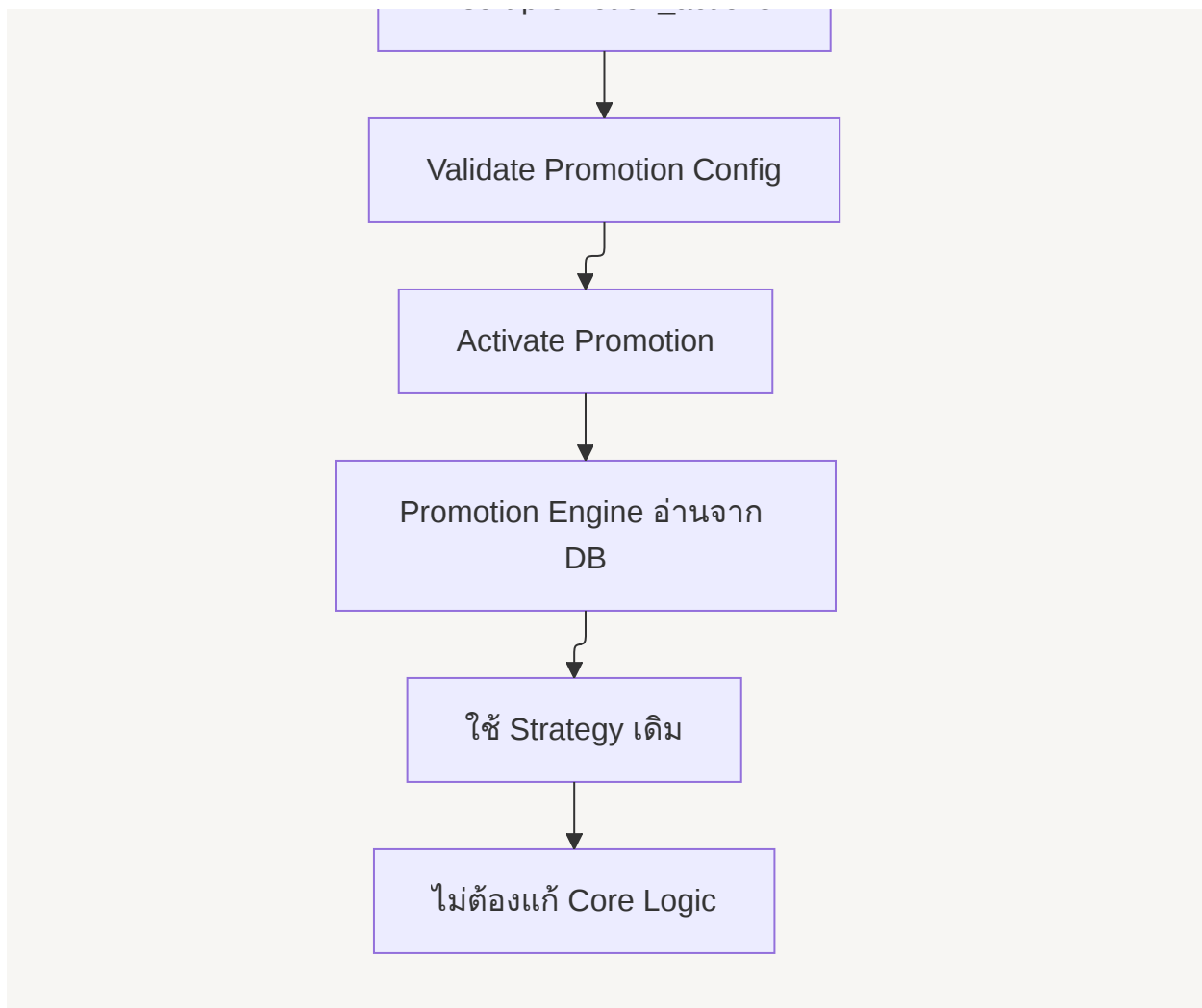
```
H --> I["Promotion Engine อ่านจาก DB"]
```

```
I --> J["ใช้ Strategy เดิม"]
```

```
J --> K["ไม่ต้องแก้ Core Logic"]
```







ตัวอย่าง:

ต้องการเพิ่ม Product 3 ลด 15%

เพิ่มแค่ config:

```
promotions.scope = ITEM
promotion_targets.target_type = PRODUCT
promotion_targets.target_id = 3
promotion_actions.action_type = PERCENTAGE_DISCOUNT
promotion_actions.value_percent = 15
```

ไม่ต้องแก้ calculation engine

7. Add New Promotion Type Flow

ภาพนี้ตอบว่า ถ้าต้องเพิ่ม promotion ใหม่ที่ไม่เคยมีมาก่อน จะ design ยังไง

flowchart TD

```
A["ต้องการเพิ่ม Promotion Type ใหม่"] --> B["Define Business Rule"]
```

```
B --> C["เพิ่ม action_type ใหม่<br/>เช่น BUY_X_GET_Y"]
```

```
C --> D["ออกแบบ Config ใน value_json"]
```

```
D --> E["สร้าง Strategy ใหม่"]
```

```
E --> F["Register Strategy เข้า Strategy Registry"]
```

```
F --> G["เพิ่ม Validation Rule"]
```

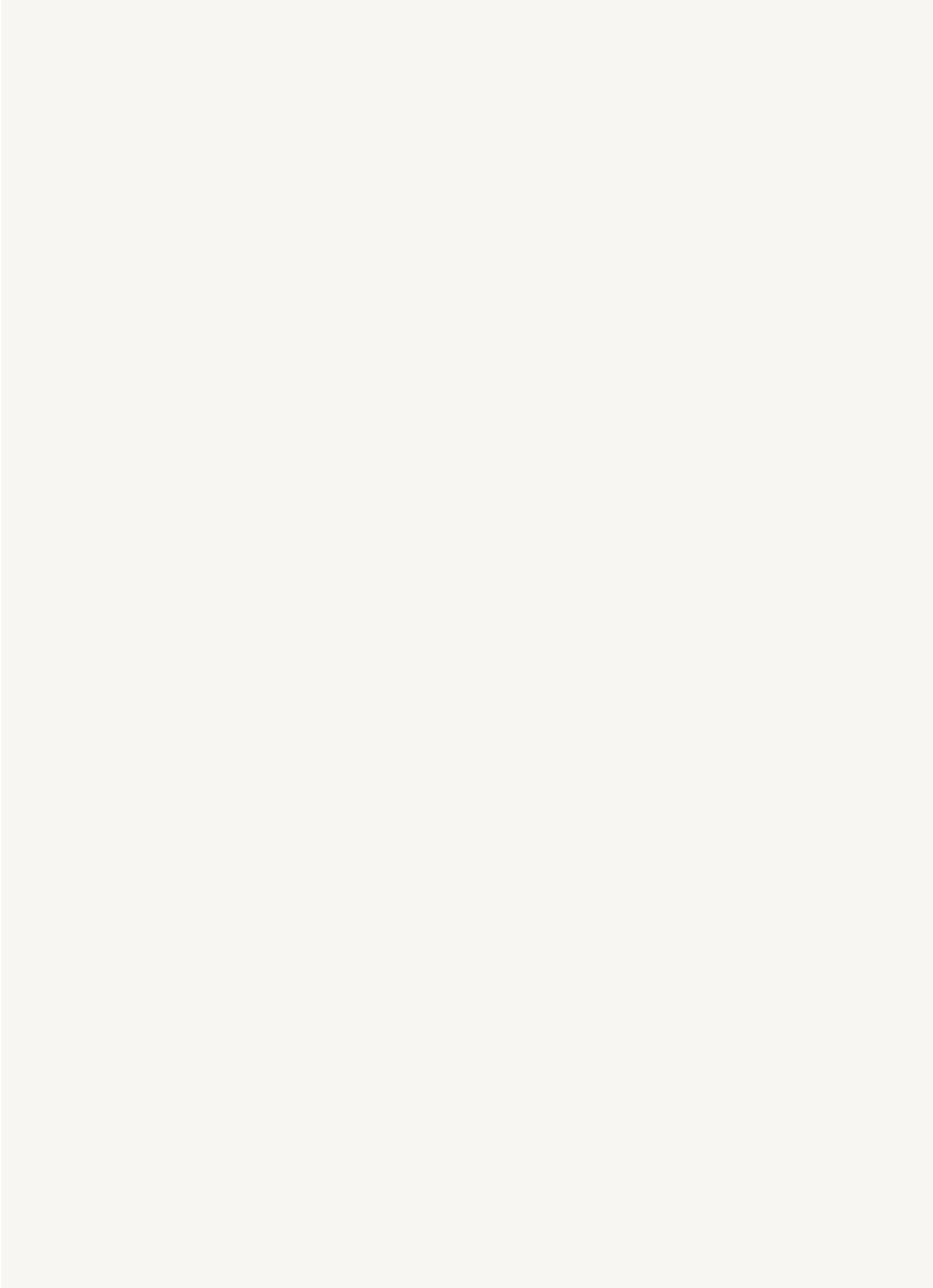
```
G --> H["เพิ่ม Test Cases"]
```

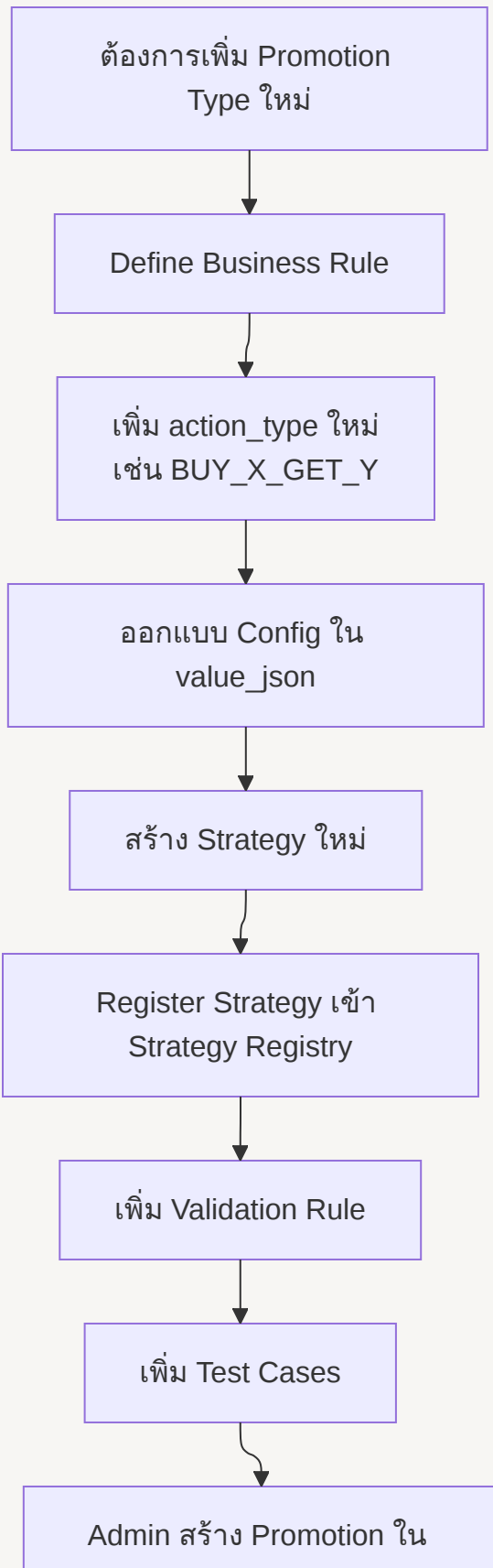
```
H --> I["Admin สร้าง Promotion ใน DB"]
```

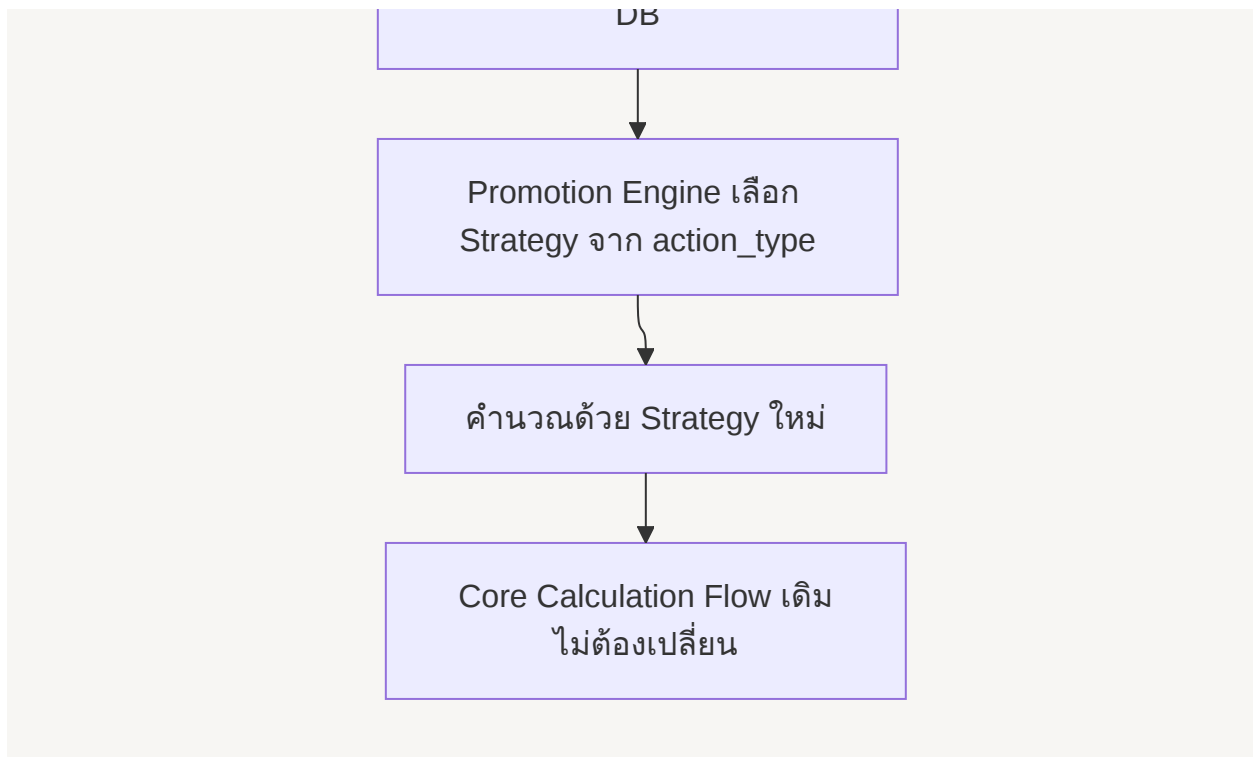
```
I --> J["Promotion Engine เลือก Strategy จาก action_type"]
```

```
J --> K["คำนวณด้วย Strategy ใหม่"]
```

```
K --> L["Core Calculation Flow เดิมไม่ต้องเปลี่ยน"]
```







ตัวอย่าง promotion ใหม่:

ซื้อ 2 แกม 1

เก็บใน DB ได้แบบนี้:

```
promotion_actions.action_type = BUY_X_GET_Y
promotion_actions.value_json = {
  "buyQuantity": 2,
  "getQuantity": 1,
  "discountPercent": 100
}
```

สิ่งที่ต้องเพิ่มใน code มีแค่:

BuyXGetYStrategy

แต่ flow หลักยังเหมือนเดิม:

Load → Evaluate → Sort → Apply Strategy → Log

8. Correct Calculation Diagram

ภาพนี้ใช้อธิบายว่า DB + Engine ช่วยให้คำนวณถูกต้องอย่างไร

flowchart TD

A["Order Items"] --> B["Load Server-side Product Price"]

B --> C["Calculate original_amount"]

C --> D["Apply Eligible Promotions"]

D --> E["Calculate calculated_discount"]

E --> F["Cap Discount"]

F --> F1["actual_discount = min(calculated_discount, remaining_amount)"]

F1 --> G["Update item/cart total"]

G --> H["final_amount < 0 ?"]

H -- "Yes" --> I["Set final_amount = 0
Record Warning"]

H -- "No" --> J["Keep final_amount"]

I --> K["Generate Breakdown"]

J --> K

K --> L["Save applied_promotions_json"]

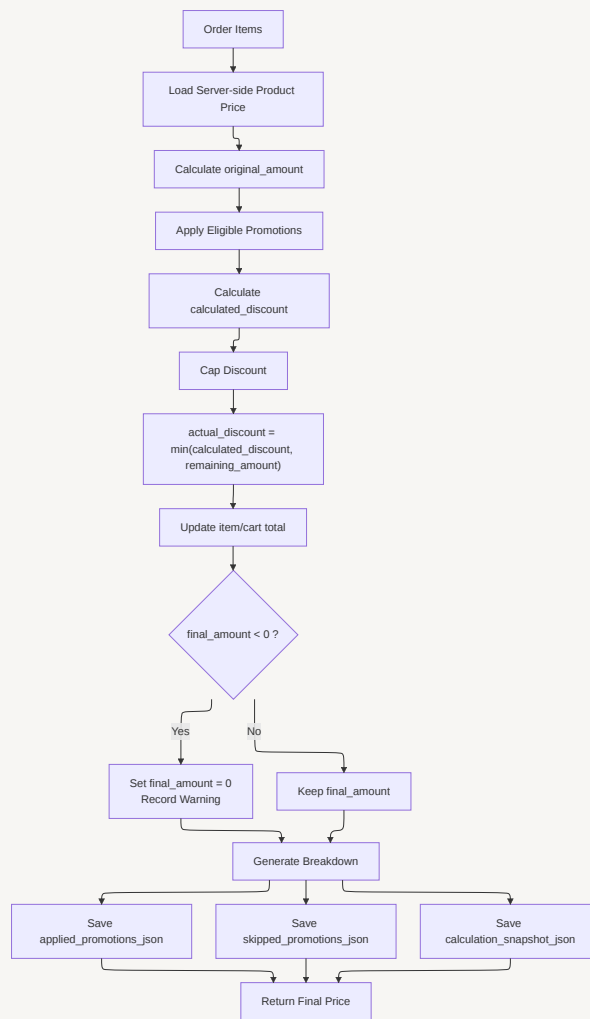
K --> M["Save skipped_promotions_json"]

K --> N["Save calculation_snapshot_json"]

L --> O["Return Final Price"]

M --> O

N --> O



กฎสำคัญเพื่อให้คำนวณถูกต้อง

1. ราคาสินค้าต้องมาจาก products table เท่านั้น
2. ห้ามใช้ราคาจาก client
3. เงินควรเก็บเป็น integer หน่วยสตางค์
4. ส่วนลดต้องไม่เกินราคาที่เหลือ
5. final_total ต้องไม่ติดลบ
6. ต้องเก็บ applied/skipped promotion ทุกครั้ง
7. ต้องมี calculation snapshot สำหรับตรวจย้อนหลัง

9. End-to-End Data Flow Diagram

ภาพนี้ใช้อธิบาย flow ตั้งแต่ customer กดคำนวณราคาไปจนได้ราคาสุทธิ

sequenceDiagram

participant C as Customer / Client

participant API as Go Fiber API

participant PS as Pricing Service

participant PE as Promotion Engine

participant DB as MySQL Database

participant LOG as Calculation Log

C->>API: POST /api/v1/pricing/calculate

API->>PS: Validate Request

PS->>DB: Load Products

DB-->>PS: Product Prices

PS->>DB: Load Active Promotions

DB-->>PS: Promotion Config

PS->>PE: Build Calculation Context

PE->>PE: Check Targets

PE->>PE: Check Conditions

PE->>PE: Sort by Scope + Priority

PE->>PE: Apply Stacking Policy

PE->>PE: Apply Strategy

PE->>PE: Cap Discount and Round

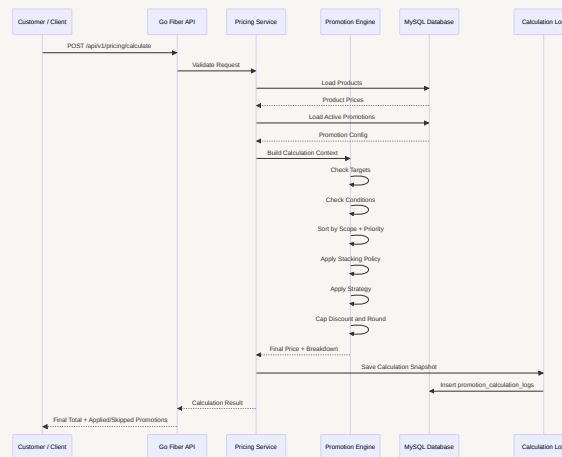
PE-->>PS: Final Price + Breakdown

PS->>LOG: Save Calculation Snapshot

LOG->>DB: Insert promotion_calculation_logs

PS-->>API: Calculation Result

API-->>C: Final Total + Applied/Skipped Promotions



10. สรุปคำตอบที่ Diagram ชัดนี้ตอบโจทย์

คำถาม	Diagram ที่ตอบ
Promotion ซ้อนกันจัดลำดับยังไง	Diagram 2, 3
เพิ่ม promotion โดยไม่กระทบ logic เดิม	Diagram 6
เพิ่ม promotion ใหม่ที่ไม่เคยมีมาก่อน	Diagram 7
ใช้ design pattern อะไร	Diagram 4
Table ยึดหยุ่นยังไง	Diagram 5
คำนวณถูกต้องได้ยังไง	Diagram 8, 9

สรุปสุดท้ายสำหรับอธิบายกับพี่:

ระบบนี้ออกแบบเป็น Rule-based Promotion Engine โดยแยก Promotion ออกเป็น Target, Condition, Action และ Policy

การคำนวณใช้ลำดับ ITEM → CART → COUPON → SHIPPING → ROUNDING
และใช้ Strategy Pattern เพื่อแยก logic ของ promotion แต่ละประเภท

ถ้าเพิ่ม promotion ที่ระบบรองรับอยู่แล้ว จะเพิ่มผ่าน database configuration ได้เลย

ถ้าเป็น promotion type ใหม่ จะเพิ่ม Strategy ใหม่และ register เข้า engine โดยไม่ต้องเปลี่ยน core calculation flow เดิม

ความถูกต้องของการคำนวณควบคุมด้วย priority, stacking policy, discount cap, final price validation

และบันทึก calculation log เพื่อ audit/replay ได้